Project Proposal: Hotel Management System

1. Justification of Project Topic

The Hotel Management System was chosen to implement complex software engineering principles within a real-world domain requiring high data integrity and multi-user synchronization. By building this system from scratch, the project demonstrates advanced Object-Oriented Programming (OOP) concepts, robust exception handling, and persistent data management, ensuring a scalable and professional architecture.

2. Project Scope and Focus Area

The primary domain is **Hospitality and Resource Management**, focusing on streamlining operations for hotel staff while providing a seamless booking experience for guests.

2.1 What the system will do:

- **Role-Based Access Control:** Provide distinct interfaces and permissions for Admin (system management), Receptionist (booking/check-in), and Guest (viewing/reservations) roles [cite: Admin.java, Receptionist.java, Guest.java].
- **Resource Management (CRUD):** Allow administrative staff to manage Rooms, Room Types, and Amenities through a standardized Manageable interface contract [cite: Manageable.java, Admin.java].
- **Dynamic Availability Engine:** Automatically calculate room availability in real-time by cross-referencing requested dates against all existing database records to prevent scheduling overlaps [cite: BookingEngine.java].
- **Automated Business Logic:** Enforce strict operational rules, including a 30% late-cancellation penalty for requests within 3 days of check-in and room capacity validation [cite: BookingEngine.java].
- **Financial Management:** Generate detailed invoices that aggregate room costs, dining packages, amenity surcharges, and valid promo code discounts [cite: BookingEngine.java, Invoice.java].

2.2 What the system will not do:

- **Real-Time Banking:** The system will not integrate with live banking APIs; it simulates payment flows through the Payable interface [cite: Payable.java].
- **External Web Services:** It is designed as a standalone desktop application with local file persistence rather than a cloud-hosted web server [cite: Database.java].

3. Project Objectives (Software Engineering Concerns)

Objective Category	Description
Reliability (Fault Tolerance)	Implement comprehensive exception handling (e.g., throws Exception) across all logic modules to ensure system stability and provide user-friendly feedback for invalid operations [cite: Manageable.java, Admin.java].
Data Integrity (Persistence)	Utilize Java Serialization to store and retrieve the entire system state (Guests, Rooms, Reservations, Invoices) from a local .dat file, ensuring consistency between sessions [cite: Database.java].
Maintainability (Interface Contract)	Enforce strict architectural patterns through the use of interfaces like Manageable and Payable, allowing for future expansion of roles and payment methods without breaking core logic [cite: Manageable.java, Payable.java].
Usability (Automation)	Automate complex manual tasks such as promo code validation, seasonal pricing adjustments, and guest recommendation algorithms based on stored preferences [cite: BookingEngine.java, PromoCode.java].

4. Technical Implementation & Requirements

- **Encapsulation:** All domain entities (Room, Guest, Reservation) utilize private fields and standardized accessor methods to protect internal state [cite: Room.java, Guest.java, Reservation.java].
- **Collections Framework:** Use dynamic ArrayList structures to manage resource lists, allowing the system to scale without predefined size constraints [cite: Database.java].
- **Temporal Logic:** Leverage the java.time API to handle all check-in/check-out comparisons and cancellation period calculations accurately [cite: BookingEngine.java]

5. Target Audience & Problem Statement

5.1 Intended Users

- **Hotel Administrators/Managers:** Users who need high-level oversight of financial performance, revenue reports, and room management.
- **Front Desk Receptionists:** Staff responsible for managing the day-to-day lifecycle of a

guest, from check-in to processing payments and handling cancellations.
- **Hotel Guests:** The end customers who require a seamless booking experience, transparent pricing (including amenities and dining), and fair cancellation policies.

5.2 Problem Solved

Traditional manual booking methods often lead to **scheduling overlaps** and **financial leakage** (uncollected cancellation fees). This system solves these issues by:

- Eliminating double-bookings through a real-time **Dynamic Availability Engine**.
- Automating complex financial calculations like 30% late-cancellation penalties and amenity-inclusive invoicing, reducing human error at the front desk.
- Providing instant financial clarity through date-range revenue reporting.

---

6. User Stories & Acceptance Criteria

User stories define the system's behavior from the perspective of its users. Each story is accompanied by strict Acceptance Criteria (AC) following the Given-When-Then format.

6.1 Customer Stories

ID	User Story	Acceptance Criteria
US-01	As a customer, I want to search for available rooms, so that I can find a suitable place to stay.	**Given:** I enter dates. **When:** I search. **Then:** Available rooms are displayed.
US-02	As a customer, I want to book a room, so that I can reserve my stay in advance.	**Given:** I select a room. **When:** I confirm booking. **Then:** Reservation is saved.
US-03	As a customer, I want to cancel my booking, so that I can change my plans if needed.	**Given:** I have a reservation. **When:** I cancel it. **Then:** It is removed/updated in the system.
US-04	As a customer, I want to see a "no rooms available" message, so that I understand availability issues.	**Given:** No rooms exist for the dates. **When:** I search. **Then:** The message appears.

6.2 Admin & Receptionist Stories

ID	User Story	Acceptance Criteria
US-05	As an admin, I want to add new rooms, so that the system reflects hotel availability.	**Given:** Room details are entered. **When:** Saved. **Then:** Room appears in system.
US-06	As an admin, I want to manage bookings, so that I can monitor operations.	**Given:** Bookings exist. **When:** I view them. **Then:** All reservations are shown.
US-07	As a receptionist, I want to create bookings for walk-ins, so that I can assist guests.	**Given:** Customer details provided. **When:** Booking created. **Then:** Saved successfully.
US-08	As a receptionist, I want to check room availability, so that I can inform customers quickly.	**Given:** dates are entered **When:** checked **Then:** available rooms are shown
US-09	As a customer, I want to see a message when no rooms are available, so that I understand availability issues.	**Given:** no rooms exist **When:** I search **Then:** a “no rooms available” message appears

7. Functional Requirements Specification

The following requirements are prioritized using the MoSCoW method (Must Have, Should Have, Could Have).

Module 1: Authentication & Access Control

- **FR-01:** Create new guest accounts with username, address, password, email, and phone (**Must Have**).
- **FR-02:** Authenticate user credentials for Guest, Admin, and Receptionist portals (**Must Have**).
- **FR-55:** Verify credentials against role-specific databases during authentication (**Must Have**).
- **FR-29:** Display "Access Denied" error for incorrect login credentials (**Must Have**).

- **FR-03:** Provide management dashboard access for authenticated Admins and Receptionists (**Must Have**).
- **FR-05:** Lock account temporarily after 5 failed login attempts (**Should Have**).
- **FR-04:** Send a password reset link to the registered email upon request (**Should Have**).

Module 2: Room Search & Inventory Management

- **FR-06:** Display current inventory of reserved and available rooms for staff (**Must Have**).
- **FR-41:** Retrieve a specific room record when searching by room number (**Must Have**).
- **FR-07:** Filter available rooms by specific type or maximum price limit (**Should Have**).
- **FR-34:** Sort all available rooms from low-to-high price (**Could Have**).
- **FR-08:** Toggle "Low to High" sorting option for search results (**Could Have**).
- **FR-35:** Recommend rooms matching the guest's saved preferences (**Could Have**).

Module 3: Booking & Customization

- **FR-10:** Display dining package options (e.g., half-board, full-board) during configuration (**Must Have**).
- **FR-11:** Update total reservation cost when the dining package is modified (**Must Have**).
- **FR-12:** Add selected amenity costs to the reservation total (**Must Have**).
- **FR-30:** Update total reservation cost when the room view is modified (**Must Have**).
- **FR-31:** Reject bookings that exceed the maximum capacity of the room type (**Must Have**).
- **FR-52:** Calculate total reservation nights from check-in and check-out dates (**Must Have**).
- **FR-09:** Suggest fixed-night stay packages during booking (**Should Have**).
- **FR-13:** Display an itemized list of all selected amenities at checkout summary (**Could Have**).

Module 4: Checkout & Payment Processing

- **FR-14:** Confirm room reservation only after successful payment processing (**Must Have**).
- **FR-15:** Support multiple payment methods during the checkout phase (**Must Have**).
- **FR-32:** Release reserved rooms back to inventory if the payment gateway declines or times out (**Must Have**).
- **FR-47:** Calculate final payable net total after subtracting promotional discounts (**Must Have**).
- **FR-48:** Update internal payment status flag to "true" upon successful transaction (**Must Have**).
- **FR-18:** Generate a detailed invoice once payment is fulfilled (**Should Have**).
- **FR-16:** Apply a percentage discount when a valid promo code is entered (**Could Have**).
- **FR-17:** Reject expired or invalid promotional codes (**Could Have**).

Module 5: Cancellation & Penalty Management

- **FR-19:** Cancel existing reservations via the guest user portal (**Must Have**).

- **FR-20:** Deduct a late cancellation penalty if cancelled within 72 hours of check-in (**Must Have**).
- **FR-21:** Charge the full remaining booking fee for mid-stay cancellations (**Must Have**).
- **FR-33:** Recalculate the final invoice total if check-out dates are modified mid-stay (**Must Have**).

Module 6: Admin Dashboard & Revenue Analytics

- **FR-23:** Calculate total generated revenue for end-of-month financial summaries (**Must Have**).
- **FR-24:** Generate a comprehensive financial report for the previous 30 days (**Must Have**).
- **FR-36:** Calculate the average room occupancy percentage for financial reports (**Must Have**).
- **FR-46:** Calculate total dining surcharges (daily rate × number of nights) (**Must Have**).
- **FR-45:** Calculate total revenue within a custom timeframe input by the admin (**Should Have**).
- **FR-22:** Apply a seasonal multiplier to base rates for high-demand date ranges (**Could Have**).

Module 7: Staff Operations & Administration

- **FR-37:** Update reservation status during guest check-in or check-out (**Must Have**).
- **FR-38:** Record manual cash payments submitted through the staff portal (**Must Have**).
- **FR-39:** Create, modify, and delete room, amenity, and room type records (**Must Have**).
- **FR-40:** Retrieve a specific guest profile by searching the username (**Must Have**).
- **FR-42:** Retrieve specific reservation or invoice records by ID (**Must Have**).
- **FR-49:** Display a comprehensive list of all registered guest usernames (**Must Have**).
- **FR-50:** Display a list of reservation IDs and associated guest usernames (**Must Have**).
- **FR-51:** Display "item not found" error for non-existent inventory entities (**Must Have**).

Module 8: Reviews & Ratings

- **FR-43:** Record a guest review and score submitted during checkout (**Should Have**).
- **FR-44:** Update the room's average rating when a new review is recorded (**Should Have**).

Module 9: Data Persistence & System State

- **FR-25:** Load all serialized system data from local storage on application startup (**Must Have**).
- **FR-26:** Save the updated system state to local storage whenever a record is modified (**Must Have**).

8. Non-Functional Requirements Specification

8.1 Product Requirements

- **Usability:** Provide smooth and intuitive transitions between screens using JavaFX Scene switching or Stack Pane layers.
- **Robustness:** Validate all user inputs (e.g., balance, date ranges) and handle errors via custom exceptions to ensure system stability.
- **Visual Fidelity:** Render all multimedia content, including room photos and videos, in high quality to maintain professional standards.
- **Efficiency:** Synchronize reservation data with the database immediately upon any transaction and reflect state changes in real-time.
- **Security:** Maintain strict data encapsulation by utilizing private fields and public getters/setters to protect internal object states.
- **Accessibility:** Design the user interface and menus for maximum accessibility across various user age groups.

8.2 Organizational Requirements

- **Technology Stack:** Develop the application using the **Java** programming language and the **JavaFX** framework for the GUI.
- **UI Implementation:** Implement at least three primary screens using **FXML** to ensure a clean separation of concerns following the MVC (Model-View-Controller) pattern.
- **Professional Styling:** Apply a global **CSS stylesheet** across the application to provide a consistent, modern, and professional aesthetic.
- **Networking:** Establish a robust **client-server architecture** using **Java Sockets** to support real-time communication features between users and the system.
- **Collaborative Development:** Build the project collaboratively using **Git/GitHub**, maintaining a clear, reviewable, and professional commit history.

8.3 External Requirements

- **Complete Documentation:** The shared GitHub repository must contain all source code, a comprehensive **UML Class Diagram**, and all associated project documentation.
- **Report Standards:** Deliver the final project report as a **.docx** file, covering detailed analysis, design decisions, and testing methodologies.
- **Demonstration:** Upload a video to **YouTube** providing a functional demo of the software, illustrating all key features and user workflows.

9. Innovation and Value Proposition

9.1 Differentiation from Existing Solutions

Unlike basic booking templates, this system integrates **Automated Business Logic** directly into the core engine.

- **Smart Overlap Detection:** Uses `java.time` logic to verify room availability not just by "status," but by cross-referencing specific date ranges against the entire reservation database.
- **Granular Cost Customization:** Allows for a multi-layered pricing model that combines base room rates, view-based surcharges, dining packages, and individual amenity costs into a single, cohesive invoice.
- **Data Persistence:** Uses a custom `Database` serialization layer to ensure all hotel records are preserved safely across system restarts without the overhead of complex external SQL servers.

9.2 Real-World Impact

This system transitions a hotel from **reactive** management to **proactive** management.

- **Financial Security:** The automated penalty system protects the hotel's bottom line during peak seasons when late cancellations otherwise result in lost revenue.
- **Operational Efficiency:** Front desk staff spend less time on manual math and more time on guest service, as the system handles all capacity and pricing validations automatically.
- **Informed Decision Making:** Managers can generate precise revenue reports between any two dates, allowing them to identify profitable trends and adjust business strategies accordingly.

.